University of Southern Britanny – UBS

Course Name:

M1-ISC-CSSE-CYBERUS - Applied Cryptography

$

Lecturer: Professor Guy GOGNIAT

LAB REPORT – 2

(AES algorithm and modes of operations)

Submitted by :

Name; Assan Jallow

Student ID: E2506147

Email: jallow.e2506147@etud.univ-ubs.fr

Lab 2: AES algorithm and modes of operations

Question 1:
Explain the AES key generation algorithm.

Answer:
AES uses a key expansion (a.k.a. key schedule) to derive a set of round keys from the original key. For AES-128, the 128-bit key is split into 4 words (32-bit each), and then expanded to 44 words. Words are generated as:

- Let Nk be the number of key words (4 for AES-128), Nb=4 (block words), and Nr=10 (rounds).
- For each i from Nk to 4(Nr+1)−1:
    - temp = w[i−1]
    - If i % Nk == 0, then temp = SubWord(RotWord(temp)) XOR Rcon[i/Nk]
        - RotWord rotates a 4-byte word by 1 byte to the left.
        - SubWord substitutes each byte with the S-box value.
        - Rcon injects a round constant in the high byte.
    - For AES-256 an extra SubWord is applied when i % Nk == 4.
    - Finally, w[i] = w[i−Nk] XOR temp.
      Grouping each 4 words gives a 128-bit round key. The cipher uses the initial AddRoundKey, 9 standard rounds, and 1 final round, each consuming one round key (hence the expanded key schedule).

Question 2:
How many subkeys are generated? is it compliant with the AES algorithm?

Answer:
AES-128 generates 11 round keys (one for the initial AddRoundKey plus one per each of the 10 rounds). In word count, that's 44 words (4 words × 11). AES-192: 13 round keys; AES-256: 15 round keys. And yes, it's compliant with the AES standard (FIPS-197).

Question 3:
Complete the three missing functions. Explain your code.

Code:

```
SBOX = [
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16
]


# e.g. def sub_bytes(), def shift_rows(), def encrypt(), etc.

# xtime for GF(2^8) multiplication
xtime = lambda a: (((a << 1) ^ 0x1B) & 0xFF) if (a & 0x80) else (a << 1)

# Convert 4x4 matrix to 128-bit integer
def matrix2text(matrix):
    text = 0
    for i in range(4):
        for j in range(4):
            text |= (matrix[i][j] << (120 - 8 * (4 * i + j)))
    return text

# AES encryption main function
def encrypt(plaintext):
    plain_state = text2matrix(plaintext)

    add_round_key(plain_state, round_keys[:4])

    for i in range(1, 10):
```

```
        round_encrypt(plain_state, round_keys[4 * i : 4 * (i + 1)])

    sub_bytes(plain_state)
    shift_rows(plain_state)
    add_round_key(plain_state, round_keys[40:])

    return matrix2text(plain_state)

# AddRoundKey
def add_round_key(s, k):
    for i in range(4):
        for j in range(4):
            s[i][j] ^= k[i][j]

# Round = SubBytes → ShiftRows → MixColumns → AddRoundKey
def round_encrypt(state_matrix, key_matrix):
    sub_bytes(state_matrix)
    shift_rows(state_matrix)
    mix_columns(state_matrix)
    add_round_key(state_matrix, key_matrix)

# SubBytes
def sub_bytes(s):
    for i in range(4):
        for j in range(4):
            s[i][j] = SBOX[s[i][j]]
    return s

# ShiftRows
def shift_rows(s):
    s[1] = s[1][1:] + s[1][:1]
    s[2] = s[2][2:] + s[2][:2]
    s[3] = s[3][3:] + s[3][:3]
    return s

# MixColumns helpers
def mix_single_column(a):
    t = a[0] ^ a[1] ^ a[2] ^ a[3]
    u = a[0]
    a[0] ^= t ^ xtime(a[0] ^ a[1])
    a[1] ^= t ^ xtime(a[1] ^ a[2])
    a[2] ^= t ^ xtime(a[2] ^ a[3])
    a[3] ^= t ^ xtime(a[3] ^ u)

def mix_columns(s):
```

```
    for i in range(4):
        mix_single_column(s[i])
    return s
```

Question 4:
Based on what you have done for encryption complete the three missing functions. Explain your code.

Answer:
The code implements part of the AES decryption process, using inverse transformations to reverse what happens in encryption. The function decrypt(ciphertext) first converts the input into a 4×4 state matrix, then performs the final decryption round by applying AddRoundKey, InvShiftRows, and InvSubBytes. After that, it goes through 9 full inverse rounds using the round_decrypt() function, which applies AddRoundKey, InvMixColumns, InvShiftRows, and InvSubBytes in that order (because decryption is the reverse of encryption). Finally, the first round key is added to recover the original plaintext state, which is then converted back to text. The helper functions like inv_shift_rows() and inv_sub_bytes() implement the inverse operations using the inverse S-box and row rotations. inv_mix_columns() reverses the column transformation using Galois Field arithmetic. Overall, the code follows the AES decryption structure, reversing each encryption step to retrieve the original plaintext.

Question 5:
What can you say about this code? compare the execution with the previous code you have written. Do you obtain the same result?

Answer:
Yes, the same result, , when I compared the AES encryption results from the code built with the example or library implementation, they produced the same ciphertext and the same decrypted plaintext. This shows that the functions implemented (SubBytes, ShiftRows, MixColumns, AddRoundKey, and Key Expansion) follow the AES standard correctly. The main difference is that the code is more step-by-step, while the library code is more optimized and faster, but both give identical results for the same key and plaintext.

Question 6:
Try other modes of operation from the library and comment the results. Explain how mode works each mode.

Answer:
I tested AES in several modes with the same key and a repeating plaintext. ECB produced identical ciphertext blocks for identical plaintext blocks (pattern leak → not secure). CBC uses a random IV and XOR chains blocks, hiding patterns (needs padding). CFB/OFB turn AES into a stream-like cipher and don't require padding. CTR encrypts a nonce||counter to make a keystream (fast,

parallelizable, no padding; nonce must never repeat). GCM adds authentication (AEAD), returning a tag that detects tampering. The results show CBC/CFB/OFB/CTR/GCM hide patterns while ECB does not; GCM is preferred when integrity is needed.

Question 7:
Build the proposed code. Explain your code

Answer:
The program implements a simple version of the ECB (Electronic Codebook) encryption process for files. It begins by opening a text file, reading the content, and dividing the data into 128-bit (16-byte) blocks. If the file size is not a multiple of 16 bytes, PKCS#7 padding is applied to complete the last block. Each block is then encrypted independently using a basic XOR operation with a 16-byte key, demonstrating the ECB concept where identical plaintext blocks produce identical ciphertext blocks. After encryption, the program decrypts each block using the same key and removes the padding. Finally, the decrypted blocks are combined to reconstruct the original text, and the result is compared with the initial file to verify that the process works correctly.